N058 Expo 앱 빌드하고 출시 준비하기_학습서

Chapter01

Expo 앱 빌드하고 출시 준비하기

Expo Go로 보는 AI 친구 에이전트 서비스의 휴대폰 앱은 내 노트북이 켜져 있고 개발 서버가 돌 때만 내 폰에서 움직입니다. 친구에게 "이 앱 써 봐"라고 보내려면 무엇이 더 필요할까요. 앱 파일은 누가 어디서 만들고, 폰에는 어떻게 들어가며, 스토어에 올리려면 무엇을 준비해야 할까요. 오늘은 Expo의 클라우드 빌드(EAS Build)로 앱 파일을 만들어 내 폰에 설치하고, 스토어에 올리기 전에 점검할 것을 정리한 뒤, 후반부에는 내가 만들고 싶은 앱을 직접 만들어 빌드까지 해 봅니다.

주관식 퀴즈

Q. [시작 질문] 지금 Expo Go로 실행하는 앱과 스토어에서 받은 앱은 무엇이 다를까요? 노트북을 껐을 때, 코드를 고쳤을 때, 친구에게 보내고 싶을 때로 나눠 떠올리는 대로 적어 보세요.

Chapter02

앱 파일과 빌드

웹 서비스는 주소를 열 때마다 서버에서 화면 코드를 받지만, 휴대폰 앱(app)은 화면 코드와 이미지, 앱 이름과 권한 같은 설정을 앱 파일 하나에 담아 폰에 설치해 두고 실행합니다. 소스 코드를 이 앱 파일로 바꾸는 과정을 빌드(build)라고 하고, 앱은 빌드를 거치지 않으면 폰에 설치할 수 없습니다. 아래 위젯은 탭 세 개로 되어 있습니다. ①에서 빌드가 왜 필요한지, ②에서 Kotlin과 Swift로 만드는 네이티브 앱은 어디서 어떻게 빌드하는지, ③에서 Expo가 그 빌드를 어떻게 대신하는지 봅니다. 탭마다 "재생"을 누르거나 "다음"으로 한 단계씩 넘기며 아래 설명을 읽습니다.

빌드가 필요한 이유와 네이티브 앱의 빌드

폰이 소스 코드를 그대로 받지 못하는 이유는 세 가지입니다. 폰의 CPU는 사람이 쓴 코드를 바로 실행하지 못해서 기계가 실행하는 형식으로 바꾸는 컴파일(compile)이 필요하고, 운영

체제는 정해진 형식의 앱 파일만 설치하며, 개발자의 키로 만든 서명(signing)이 없는 파일은 설치를 거부합니다. 폰과 스토어는 서명을 보고 누가 만든 앱인지, 같은 개발자가 낸 새 버전인지 확인합니다.

네이티브(native) 앱은 이 세 가지를 플랫폼 회사가 만든 도구로 합니다. Android는 Gradle과 Android SDK가 Kotlin 코드를 컴파일하고 APK나 AAB로 묶은 뒤 내가 만든 서명 키로 서명하며, 이 도구는 Windows, Mac, Linux에서 모두 돌아갑니다. iOS는 Xcode가 Swift 코드를 컴파일하고 IPA로 묶은 뒤 Apple이 발급한 인증서로 서명하는데, Xcode는 Mac에서만 돌아갑니다. 두 플랫폼은 언어도 도구도 달라서 네이티브로 만들면 같은 앱을 두 번 만들고 두 번 빌드합니다.

	iOS	**Android**
앱 파일	IPA	APK(직접 설치용), AAB(스토어 제출용)
컴파일하는 도구	Xcode. Mac에서만 실행됨	Gradle과 Android SDK. Windows, Mac, Linux에서 실행됨
서명	Apple이 발급한 인증서와 프로비저닝 프로필	개발자가 만든 서명 키(keystore)
스토어 밖 설치	기본적으로 막혀 있음	사용자가 허용하면 APK를 직접 설치할 수 있음

Expo가 빌드를 대신하는 방식

Expo 앱은 내 코드(TypeScript)와, 화면을 그리고 카메라나 알림 같은 기기 기능을 쓰는 네이티브 부분으로 나뉩니다. 내 코드는 JavaScript로 실행되므로 컴파일하지 않고 Metro가 번들(bundle) 하나로 묶기만 합니다. 컴파일이 필요한 네이티브 부분은 두 가지 방법으로 내 노트북 밖으로 옮깁니다. 개발할 때 쓰는 Expo Go는 Expo 팀이 자주 쓰는 네이티브 기능을 미리 빌드해 스토어에 올려 둔 앱이라서, 노트북은 번들만 보내면 됩니다. 다른 사람에게 줄 앱 파일은 EAS Build가 만듭니다. EAS는 `app.json` 으로 `android/`, `ios/` 폴더를 새로 만든 뒤 Linux 서버와 Mac 서버에서 컴파일하고, Expo 계정에 보관한 키로 서명합니다. 그래서 내 노트북에는 Node.js만 있으면 되고, Windows 노트북으로도 iOS 앱 파일을 만들 수 있습니다.

EAS의 클라우드 서버가 받는 것은 저장소에 있는 파일뿐이라, 커밋하지 않은 `.env` 의 값은 빌드에 들어가지 않습니다. 빌드가 실패하면 빌드 로그에서 어느 단계에서 멈췄는지부터 보면 원인을 좁힐 수 있습니다. 패키지 설치에서 멈췄다면 `package.json` 의 버전 충돌을, 네이티브 컴파일에서 멈췄다면 네이티브 라이브러리나 `app.json` 설정을 먼저 의심합니다.

버전 번호와 업데이트

앱 파일에는 번호가 두 개 붙습니다. 하나는 사용자에게 보이는 버전(`app.json` 의 `version`, 예: 1.0.0)이고, 다른 하나는 스토어가 파일을 구분하는 빌드 번호(Android `versionCode`, iOS `buildNumber`)입니다. 스토어는 이미 올린 빌드 번호와 같거나 작은 파일을 받지 않으므로, 스토어에 올릴 때마다 빌드 번호가 커져야 합니다.

웹은 배포하면 모든 사용자가 새 버전을 쓰지만, 앱은 사용자가 업데이트해야 바뀌고 업데이트하지 않은 폰에는 옛 버전이 계속 남습니다. 그래서 앱을 내놓은 뒤에는 옛 버전 화면이 새 백엔드를 부르는 상황을 늘 생각해야 하고, 앱이 쓰는 백엔드 API의 응답 모양은 옛 앱이 읽지 못하게 함부로 바꾸지 않습니다. 반대로 백엔드 코드, 데이터베이스, 서버의 프롬프트처럼 앱 파일 밖에 있는 것은 고쳐도 다시 빌드할 필요가 없습니다.

객관식 퀴즈

Q. 스토어에 올린 AI 친구 앱에서 두 가지를 고치려고 합니다. (가) 서버에 있는 에이전트의 프롬프트를 바꿔 말투를 부드럽게 한다. (나) 앱 화면의 전송 버튼 문구를 바꾼다. 앱을 다시 빌드해야 하는 것은 무엇인가요?

- 1. (가)와 (나) 모두
- 2. (가)만
- 3. (나)만
- 4. 둘 다 다시 빌드하지 않아도 된다

- •

  1. (가)와 (나) 모두

- •

  - . (가)만

- •

  3. (나)만

- •

  4. 둘 다 다시 빌드하지 않아도 된다

Chapter03

같은 코드가 폰에 들어가는 세 경로

섹션 2에서 본 것처럼 Expo 앱은 내 코드와 네이티브 부분으로 되어 있습니다. Expo Go는 네이티브 부분을 미리 컴파일해 스토어에 올려 둔 앱이라서, 내 노트북의 개발 서버가 내 코드를 번들로 만들어 보내 주기만 하면 바로 실행됩니다. 그래서 코드를 저장하면 폰 화면이

곧바로 바뀝니다. 대신 개발 서버가 꺼지면 앱도 멈추고, 같은 와이파이나 tunnel로 연결된 내 폰에서만 열 수 있습니다.

다른 사람이 받을 수 있는 앱을 만들려면 내 코드와 네이티브 부분을 하나의 앱 파일로 묶는 빌드를 해야 합니다. Android 앱 파일은 바로 설치하는 APK와 스토어에 올리는 AAB가 있고, iPhone은 서명된 iOS 앱 파일이 필요합니다. 빌드에는 원래 Android Studio와 Xcode가 필요하지만, Expo의 EAS Build는 이 일을 Expo의 클라우드 서버가 대신 합니다. 프로젝트를 올리면 서버가 앱 파일을 만들어 설치 링크를 돌려주므로, 내 노트북에는 Node.js만 있으면 됩니다. 만든 파일을 스토어에 올리는 일은 제출(submit)이라고 하고, EAS Submit이나 스토어 웹 화면으로 합니다.

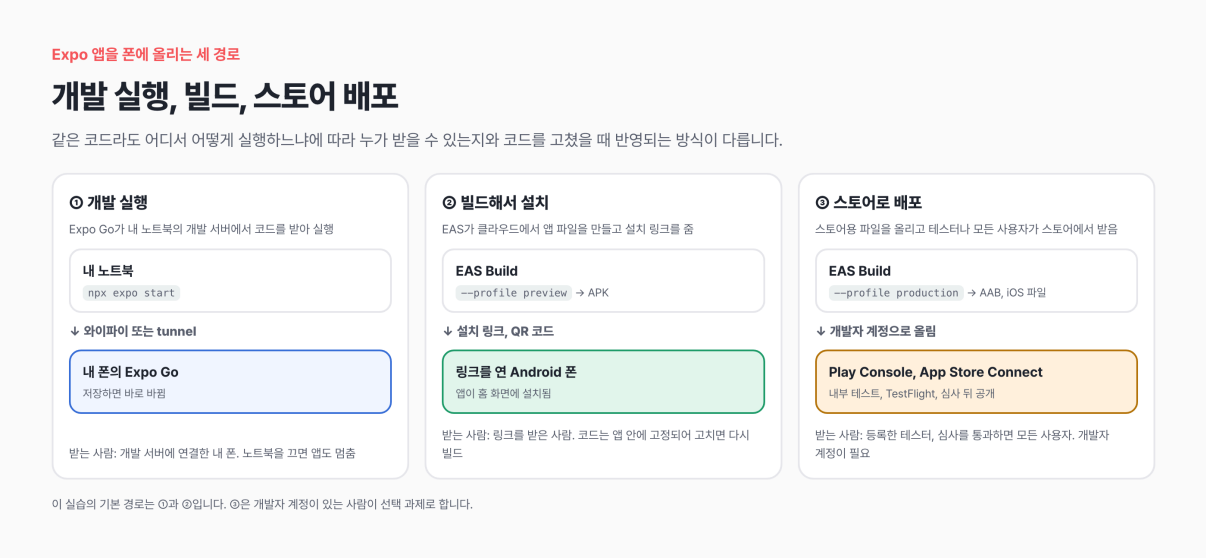


그림: Expo 앱을 폰에 올리는 세 경로: 개발 실행, 빌드해서 설치, 스토어로 배포

그림의 ②는 빌드한 앱 파일을 링크로 직접 설치하는 경로입니다. 스토어를 거치지 않으므로 개발자 계정이 필요 없고, 링크를 받은 Android 폰이면 누구나 설치할 수 있습니다. iPhone은 스토어 밖에서 앱을 설치하는 것이 막혀 있어서, 이 경로로 설치하려면 유료 Apple 개발자 계정에 그 iPhone을 미리 등록해야 합니다. 그래서 오늘 실습에서 iPhone 수강생은 ①의 Expo Go로 확인하고, ②는 Android 폰을 가진 사람과 짝을 지어 함께 합니다. ③은 Google Play나 Apple 개발자 계정이 있는 사람이 선택 과제로 해 봅니다.

빌드 프로필과 서명

어떤 앱 파일을 어떻게 만들지는 저장소의 `eas.json` 에 프로필로 적어 둡니다. 모바일 템플릿에는 프로필이 두 개 있습니다.

프로필	만드는 파일	쓰는 곳
`preview`	Android APK, 내부 배포(internal distribution)	설치 링크로 내 폰이나 팀원 폰에 바로 설치
`production`	Android AAB, iOS 스토어용 파일. 빌드할 때마다 버전 번호를 자동으로 올림	스토어에 제출

앱 파일에는 개발자가 만든 서명이 들어갑니다. 스토어는 처음 올린 앱 파일의 서명 키를 기억해 두고, 그 뒤에는 같은 키로 서명한 파일만 같은 앱의 새 버전으로 받습니다. 서명 키를 잃어버리거나 남에게 넘어가면 앱을 업데이트할 수 없거나 남이 내 앱인 척 올릴 수 있어서, 이 키는 시크릿처럼 다뤄야 합니다. EAS는 처음 빌드할 때 서명 키를 만들어 Expo 계정에 보관하고 빌드마다 꺼내 씁니다. 그래서 키 파일이 내 노트북이나 저장소에 생기지 않고, 저장소의 `.gitignore` 에도 키 파일 형식(`*.jks`, `*.p12`, `*.p8`)을 막아 두었습니다.

빌드에 들어가는 설정값

개발할 때는 저장소의 `.env` 에 적은 `EXPO_PUBLIC_API_URL` (백엔드 주소)을 개발 서버가 읽어 앱에 넣어 줍니다. 그런데 `.env` 는 커밋하지 않는 파일이라 빌드할 때 Expo 클라우드로 올라가지 않습니다. 그래서 빌드에 쓸 값은 `eas.json` 의 프로필마다 `env` 에 따로 적습니다. 이 값을 빠뜨리면 빌드는 성공하지만, 설치한 앱을 열었을 때 "백엔드 주소가 비어 있습니다"가 뜹니다.

`EXPO_PUBLIC_` 로 시작하는 값은 앱 파일 안에 그대로 들어가므로, 앱을 받은 사람 누구나 꺼내 볼 수 있습니다. 백엔드 주소처럼 공개돼도 되는 값만 넣고, LLM API 키나 Supabase secret key 같은 시크릿은 앱에 넣지 않습니다. 앱이 쓰는 Supabase 주소와 publishable key 는 백엔드 `/api/config` 에서 받아 오도록 템플릿을 만들어 두었습니다.

앱 이름과 패키지 이름

`app.json` 의 `name` 은 폰 홈 화면에 보이는 앱 이름이고, `android.package` 와 `ios.bundleIdentifier` 는 스토어와 폰이 앱을 구분하는 이름입니다. 보통 `kr.회사이름.앱이름` 처럼 도메인을 뒤집은 모양으로 짓습니다. 같은 패키지 이름의 앱은 한 폰에 하나만 설치되므로, 템플릿의 `com.example.harufriend` 를 그대로 두면 같은 이름으로 만든 다른 사람의 앱과 부딪힙니다. 스토어에 한 번 올린 패키지 이름은 바꿀 수 없고, 바꾸면 완전히 다른 앱이 됩니다. 그래서 빌드하기 전에 내 이름이나 별칭을 넣은 패키지 이름을 정합니다.

Expo Go와 개발용 빌드

Expo Go 안에는 자주 쓰는 네이티브 기능만 들어 있습니다. Expo Go에 없는 네이티브 라이브러리(앱 안 결제, 일부 로그인 SDK 등)를 쓰거나, Android에서 서버가 보내는 푸시 알

림을 받으려면 개발용 빌드(development build)가 필요합니다. 개발용 빌드는 내 앱 전용으로 네이티브 부분까지 다시 컴파일한 실행 앱이라서, 한 번 설치해 두면 Expo Go처럼 개발 서버에서 코드를 받아 바로바로 바뀝니다. 다만 네이티브 라이브러리를 더하거나 `app.json` 의 권한을 바꿀 때마다 다시 빌드해야 합니다.

바꾼 것	Expo Go	개발용 빌드	`preview` 빌드
화면, 문구, 동작(JavaScript)	저장하면 반영	저장하면 반영	다시 빌드
네이티브 라이브러리, 권한	Expo Go에 있는 것만	다시 빌드	다시 빌드
노트북 없이 실행	안 됨	안 됨	됨

오늘은 Expo Go와 `preview` 빌드만 씁니다. Expo Go도 보통 최신 Expo SDK 하나만 지원하므로, 프로젝트의 SDK 버전(`package.json` 의 `expo`)이 폰의 Expo Go와 맞지 않으면 앱이 열리지 않습니다. 이때는 SDK를 올리거나(`npx expo install expo@latest --fix`) 강사에게 맞는 버전을 확인합니다.

객관식 퀴즈

Q. `preview` 빌드로 APK를 만들어 폰에 설치했는데, 앱을 열자 "백엔드 주소가 비어 있습니다"가 떴습니다. Expo Go로 실행할 때는 잘 됐습니다. 가장 먼저 확인할 곳은 어디인가요?

• 1. 폰의 와이파이 설정
• 2. `eas.json` 의 `preview` 프로필 `env` 에 `EXPO_PUBLIC_API_URL` 이 내 백엔드 주소로 들어 있는지
• 3. `.gitignore` 에서 `.env` 를 지웠는지
• 4. Expo Go의 SDK 버전**제출**

- •

  1. 폰의 와이파이 설정

- •

  - . `eas.json` 의 `preview` 프로필 `env` 에 `EXPO_PUBLIC_API_URL` 이 내 백엔드 주소로 들어 있는지

- •

  3. `.gitignore` 에서 `.env` 를 지웠는지

- •

  4. Expo Go의 SDK 버전

📚 빌드와 제출 참고 자료

- Expo: EAS Build: 클라우드 빌드가 무엇을 하고 어떤 파일을 만드는지 정리한 공식 문서입니다.
- Expo: Internal distribution: `preview` 처럼 설치 링크로 앱을 나눠 주는 내부 배포의 동작과 iPhone 기기 등록 조건이 나옵니다.
- Expo: Development builds: Expo Go와 개발용 빌드의 차이를 볼 수 있습니다.

Chapter04

시작 상태 맞추기

이 교안의 실습은 아래 상태가 갖춰져 있어야 진행할 수 있습니다. 모바일 템플릿은 friend-agent-mobile-template 저장소이고, Use this template으로 내 저장소를 만들어 내려받습니다. 백엔드 주소는 내가 배포한 에이전트 서비스의 staging 주소를 쓰고, 없으면 강사가 알려 주는 시연용 주소를 씁니다. 실습은 Android 폰을 기준으로 합니다. iPhone 수강생은 Expo Go로 확인하고, 빌드한 앱의 설치는 Android 폰을 가진 사람과 짝을 지어 짝의 폰으로 합니다.

필요한 상태	확인할 것	없으면
Node.js 20 이상	`node -v`	nodejs.org에서 LTS 버전 설치
모바일 템플릿으로 만든 내 저장소와 패키지	저장소 폴더에 `node_modules` 가 있음	Use this template → 내려받기 → `npm install`
`.env` 의 백엔드 주소	브라우저로 `<백엔드 주소>/health` 가 열림	`.env.example` 을 `.env` 로 복사해 주소 넣기
Expo Go로 실행되는 앱	`npx expo start` 뒤 폰에서 캐릭터 화면이나 대화 화면이 보임	모바일 템플릿 README의 "내 폰에서 실행하기"
Expo 계정(무료)	expo.dev에 로그인됨	expo.dev에서 가입. 백엔드 저장소와 같은 GitHub 계정으로 가입하면 관리가 편함

직접 실행할 명령: 시작 상태 확인 (모바일 저장소 폴더에서)

```
node -v
npm -v
npx expo-doctor
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
이 저장소가 Expo 앱을 빌드할 수 있는 상태인지 확인해 줘. node 와 npm 버전, package.json 의 expo 버전, .env 에 EXPO_PUBLIC_API_URL 이 있는지(값은 출력하지 마),
npx expo-doctor 결과를 보여 줘. 확인 명령만 실행하고 아무것도 설치하거나 바꾸지 마.
```

앱 이름, 패키지 이름, 빌드 설정값 정하기

빌드하기 전에 세 가지를 정해 파일에 적습니다. 앱 이름은 폰에서 내 앱을 알아볼 수 있게 짓고(예: `하루친구 지수`), 패키지 이름은 다른 수강생과 겹치지 않게 내 별칭을 넣습니다(예: `kr.study.harufriend.jisu`, 영문 소문자와 숫자와 점만). 빌드에 넣을 백엔드 주소는 `.env` 에 넣은 주소와 같습니다.

직접 확인할 곳: app.json 과 eas.json 고치기 (편집기)

1. `app.json` 의 `name` 을 내 앱 이름으로 바꿉니다.
2. `app.json` 의 `android.package` 와 `ios.bundleIdentifier` 를 내 패키지 이름으로 바꿉니다. 두 값은 같게 둡니다.
3. `eas.json` 의 `preview` 와 `production` 프로필 `env` 에 있는 `EXPO_PUBLIC_API_URL` 을 내 백엔드 주소로 바꿉니다. 끝에 `/` 를 붙이지 않습니다.
4. `npx expo start` 로 Expo Go에서 다시 열어 앱이 그대로 도는지 확인합니다. `app.json` 을 바꾼 뒤에는 개발 서버를 껐다 켭니다.

코딩 에이전트에게 맡길 때: 앱 이름, 패키지 이름, 빌드 설정값

```
app.json 의 name 을 [앱 이름], android.package 와 ios.bundleIdentifier 를 [패키지 이름] 으로 바꿔 줘.
eas.json 의 preview 와 production 프로필 env 의 EXPO_PUBLIC_API_URL 을 [백엔드 주소] 로 바꿔 줘.
다른 줄은 바꾸지 말고, 바꾸기 전과 후를 나란히 보여 줘.
```

실행 전에 바뀌는 줄이 이 다섯 값뿐인지 봅니다. 패키지 이름은 스토어에 올리면 바꿀 수 없으므로, 에이전트가 임의로 짓지 않고 내가 정한 값이 들어갔는지 확인합니다.

Expo 계정에 연결하고 빌드하기

이제 EAS로 빌드합니다. `eas` 명령은 설치하지 않고 `npx eas-cli@latest` 로 그때그때 최신 버전을 받아 씁니다. `login` 으로 내 Expo 계정에 로그인하고, `build:configure` 로 이 저장소를 내 Expo 계정의 프로젝트로 연결합니다. 연결하면 `app.json` 에 프로젝트 ID가 추가되고, `eas.json` 이 이미 있으므로 그 내용을 씁니다. 어떤 플랫폼을 설정할지 물으면 Android를 고르거나 모두를 고릅니다.

직접 실행할 명령: Expo 계정 연결과 Android preview 빌드 (모바일 저장소 폴더에서)

```
npx eas-cli@latest login
npx eas-cli@latest build:configure

# 클라우드에서 Android APK를 만든다. 처음이면 서명 키를 만들지 묻는다 → Yes
npx eas-cli@latest build --profile preview --platform android
```

코딩 에이전트에게 맡길 때: Android preview 빌드

```
이 저장소를 내 Expo 계정 프로젝트로 연결하고(build:configure), preview 프로필로 Android 빌드를 시작해 줘.
eas 는 npx eas-cli@latest 로 실행해 줘. 빌드 전에 npx tsc --noEmit 과 npx expo-doctor 를 먼저 돌려서 문제가 없을 때만 빌드해 줘.
서명 키를 만들지 물으면 EAS가 만들어 보관하는 쪽으로 진행하고, 키 파일을 내 컴퓨터에 내려받지 마.
```

로그인은 브라우저나 터미널에서 내가 직접 합니다. 비밀번호를 에이전트 대화에 붙여 넣지 않습니다. 빌드 명령을 승인하기 전에 프로필이 `preview`, 플랫폼이 `android` 인지 봅니다. 무료 플랜은 월 빌드 횟수가 정해져 있어서(2026년 10월 기준 월 30회, 낮은 우선순위 대기열), 타입 검사나 설정 검사에서 걸러질 실수로 빌드 한 번을 쓰지 않게 검사를 먼저 돌립니다.

결과 해석: 클라우드에서 만든 앱 파일

명령은 프로젝트를 Expo 서버로 올린 뒤 대기열에 들어갔다는 안내와 빌드 페이지 주소를 보여 줍니다. 무료 플랜은 대기열이 길어서 시작하기까지 몇 분에서 수십 분이 걸릴 수 있고, 시작한 뒤에도 10분 안팎이 걸립니다. 터미널을 기다리지 않아도 빌드는 서버에서 계속되므로, 빌드 페이지에서 진행 단계와 로그를 볼 수 있습니다. 빌드가 끝나면 터미널과 빌드 페이지에 설치 링크와 QR 코드가 나옵니다. 처음 빌드에서 서명 키를 만들겠다고 답했다면, 빌드 페이지의 Credentials 항목에 Android 서명 키가 Expo 계정에 보관된 것으로 나옵니다. 내 노트북에는 키 파일이 생기지 않았는지도 확인합니다.

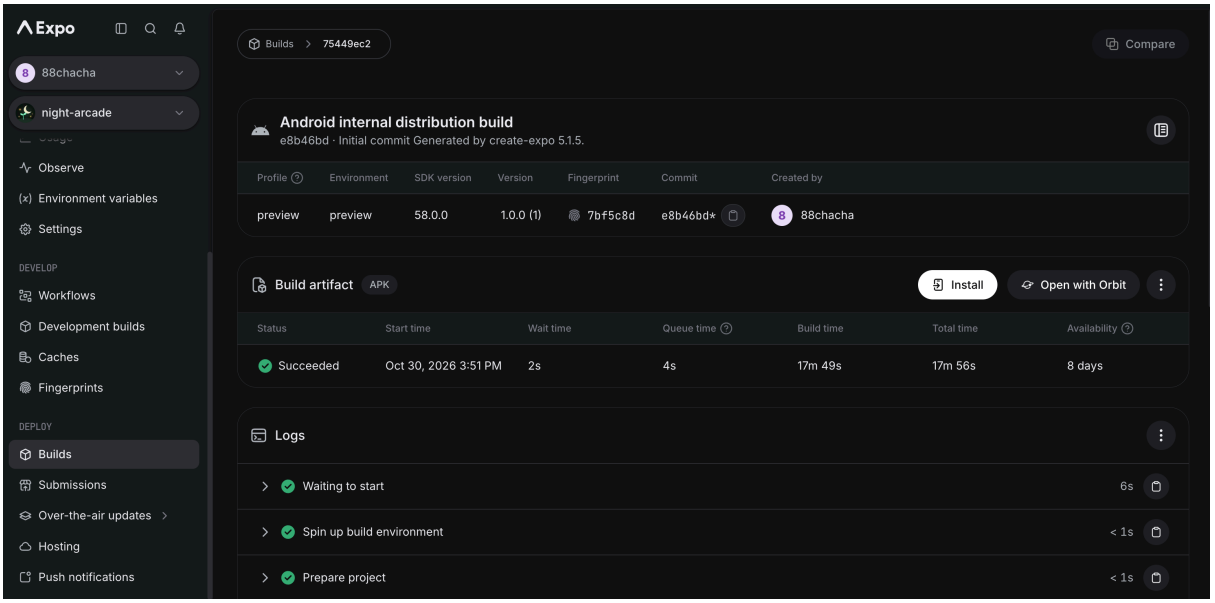


그림: expo.dev > 내 프로젝트 > Builds > 방금 만든 빌드: Android preview 빌드가 Finished 로 끝나고 Install 버튼과 QR 코드가 보이는 모습

직접 확인할 곳: 내 Android 폰에 설치하기

1. 폰으로 설치 링크를 열거나 QR 코드를 찍어 APK를 내려받습니다.
2. "출처를 알 수 없는 앱" 설치를 허용할지 물으면 이번만 허용합니다. Play 프로텍트가 "인식하지 못하는 앱"이라고 경고하면 내용을 읽고 그래도 설치를 고릅니다. 스토어를 거치지 않은 앱이라 나오는 경고입니다.
3. 홈 화면에 내가 정한 앱 이름으로 앱이 생겼는지 보고, 노트북의 개발 서버를 끈 상태에서 앱을 엽니다.
4. 캐릭터를 만들고 메시지를 보내 답이 오는지 봅니다.

결과 해석: 노트북 없이 도는 앱

개발 서버를 꺼도 앱이 열리고 대화가 됩니다. 내 코드가 앱 파일 안에 번들로 들어 있고, 앱이 직접 백엔드 주소로 요청을 보내기 때문입니다. 처음 열면 Expo Go에서 만든 캐릭터와 대화가 보이지 않고 캐릭터 화면부터 나오는데, 설치한 앱은 Expo Go와 저장 공간이 따로라서 새 게스트 사용자로 로그인하기 때문입니다. 데이터가 사라진 것이 아니라 다른 사용자가 된 것입니다. 이제 앱의 문구 하나를 고쳐 보면, Expo Go에서는 저장하자마자 바뀌지만 설치한 앱은 그대로입니다. 설치한 앱에 반영하려면 다시 빌드해야 하고, 그때마다 빌드 횟수를 하나 씁니다.

iPhone에서 빌드한 앱 실행하기

iPhone은 스토어 밖에서 받은 앱 파일을 설치할 수 없어서, Android처럼 설치 링크로 APK를 받아 까는 방법이 없습니다. 내 iPhone에서 빌드한 앱을 직접 실행해 보려면 Mac에 Xcode를 설치하고 iPhone을 케이블로 연결한 뒤, 내 Mac에서 앱을 빌드해 그 iPhone에 바로 설치합니다. Expo 프로젝트에서는 `npx expo run:ios --device` 가 Xcode로 빌드하고 연결된 iPhone에 설치하는 일을 함께 합니다. 무료 Apple 계정으로도 설치할 수 있지만 그렇게 설치한 앱은 7일 뒤에 열리지 않아 다시 설치해야 하고, iPhone 설정에서 개발자 모드를 켜야 합니다. 이 방법은 Mac이 있는 사람만 쓸 수 있으므로, 오늘 실습에서 iPhone 수강생은 Expo Go로 확인하는 것을 기본으로 합니다.

직접 실행할 명령: Mac에서 연결한 iPhone에 설치하기 (선택, Mac과 Xcode가 있을 때)

```
# iPhone을 케이블로 연결하고, 처음이면 iPhone에서 '이 컴퓨터를 신뢰' 를 누른다
npx expo run:ios --device

# 개발 서버 없이도 열리는 앱으로 설치할 때 (번들을 앱 안에 넣는다)
npx expo run:ios --device --configuration Release
```

`run:ios` 는 기본으로 개발용 구성(Debug)으로 빌드하므로, 설치한 앱이 노트북의 개발 서버에서 코드를 받아 옵니다. 노트북 없이 도는지 확인하려면 `--configuration Release` 를 붙여 번들을 앱 안에 넣습니다. 이 명령은 `ios/` 폴더를 새로 만드는데, 이 폴더는 `app.json` 으로 언제든 다시 만들 수 있으므로 커밋하지 않고 직접 고치지도 않습니다.

스토어에 올리는 빌드

오늘 만든 APK는 링크로 내 폰이나 팀원 폰에 설치해 보는 용도입니다. Google Play는 새 앱을 APK가 아니라 AAB(Android App Bundle)로만 받습니다. AAB는 스토어가 받아서 사용자 기기에 맞는 APK로 나눠 내려보내는 형식이라, AAB 파일을 폰에 바로 설치할 수는 없습니다. 그래서 스토어에 올릴 빌드는 `preview` 가 아니라 `production` 프로필로 만듭니다. 템플릿의 `production` 프로필은 Android에서는 AAB를, iOS에서는 App Store에 제출할 파일을 만들고, 빌드할 때마다 빌드 번호를 자동으로 올립니다. 섹션 2에서 본 것처럼 스토어는 이미 올린 빌드 번호와 같거나 작은 파일을 받지 않기 때문입니다.

직접 실행할 명령: 스토어에 올릴 빌드 (개발자 계정이 있을 때)

```
# Google Play에 올릴 AAB
npx eas-cli@latest build --profile production --platform android

# App Store에 올릴 iOS 빌드 (유료 Apple 개발자 계정 필요)
npx eas-cli@latest build --profile production --platform ios
```

`production` 빌드에도 `eas.json` 의 `production` 프로필 `env` 에 적은 백엔드 주소가 들어갑니다. 스토어에 올릴 앱이라면 staging이 아니라 운영(production) 백엔드 주소를 넣어야 하고, 실행 전에 프로필 이름과 이 주소를 함께 확인합니다. 스토어에 파일을 올리고 테스트 단계를 거치는 흐름은 섹션 5에서 봅니다.

코딩 에이전트로 빌드 상태 보기

빌드 상태와 로그는 expo.dev 웹 화면에서도 보지만, 코딩 에이전트에게 물어볼 수도 있습니다. 에이전트는 앞에서 로그인한 EAS CLI의 `npx eas-cli@latest build:list` 와 `build:view` 명령으로 내 계정의 빌드 목록, 상태, 설치 링크, 로그 위치를 조회합니다. 앱 폴더에 Expo 공식 스킬이 설치되어 있으면 에이전트가 Expo 팀이 정리한 빌드 설정과 오류 대응 안내를 읽고 조회하므로, 실패한 단계를 더 정확하게 짚습니다. 스킬이 없다면 아래 명령으로 앱 폴더에 설치합니다. 조회 명령은 빌드를 새로 시작하지 않아 빌드 횟수를 쓰지 않습니다.

직접 실행할 명령: Expo 스킬 설치 (앱 폴더에서, 없을 때만)

```
# Claude Code
claude plugin install expo@claude-plugins-official --scope project

# 그 밖의 코딩 에이전트
npx skills@latest add expo/skills --skill '*'
```

코딩 에이전트에게 맡길 때: 빌드 상태와 로그 확인

```
npx eas-cli@latest 로 내 Expo 계정에서 가장 최근 Android 빌드의 상태, 걸린 시간, 설치 링크를 알려
줘.
실패했다면 로그에서 실패한 단계와 원인이 되는 줄을 찾아 설명해 줘. 다시 빌드하기 전에 무엇을 고칠지 먼저 알
려 주고, 빌드는 새로 시작하지 마.
```

객관식 퀴즈

Q. 설치한 앱의 버튼 문구 하나를 고친 뒤 Expo Go에서는 바로 바뀌었는데, 폰에 설치한 앱은 그대로입니다. 가장 알맞은 설명은 무엇인가요?

- 1. 설치한 앱이 백엔드에서 화면을 받아 오는데 백엔드를 다시 배포하지 않았다
- 2. 설치한 앱에는 빌드할 때의 코드가 번들로 들어 있어서, 바꾼 코드를 반영하려면 다시 빌드해야 한다
- 3. Expo Go의 캐시가 설치한 앱에 남아 있다
- 4. 앱 이름이 Expo Go와 달라서 개발 서버에 연결되지 않았다

- •

1. 설치한 앱이 백엔드에서 화면을 받아 오는데 백엔드를 다시 배포하지 않았다

- •
  - . 설치한 앱에는 빌드할 때의 코드가 번들로 들어 있어서, 바꾼 코드를 반영하려면 다시 빌드해야 한다
- •

3. Expo Go의 캐시가 설치한 앱에 남아 있다

- •

4. 앱 이름이 Expo Go와 달라서 개발 서버에 연결되지 않았다

Chapter05

스토어 배포 단계

스토어에 앱을 올리려면 개발자 계정이 필요합니다. Google Play 개발자 계정은 등록비를 한 번 내고, Apple 개발자 계정은 해마다 회비를 냅니다. 계정이 있으면 앱을 만들고, `production` 프로필로 빌드한 파일을 올린 뒤 테스트 단계를 거쳐 공개합니다. 테스트 단계에 올린 앱은 저절로 공개되지 않습니다. 다음 단계로 넘기는 버튼을 사람이 누르고, 필요한 심사를 통과해야 넘어갑니다.

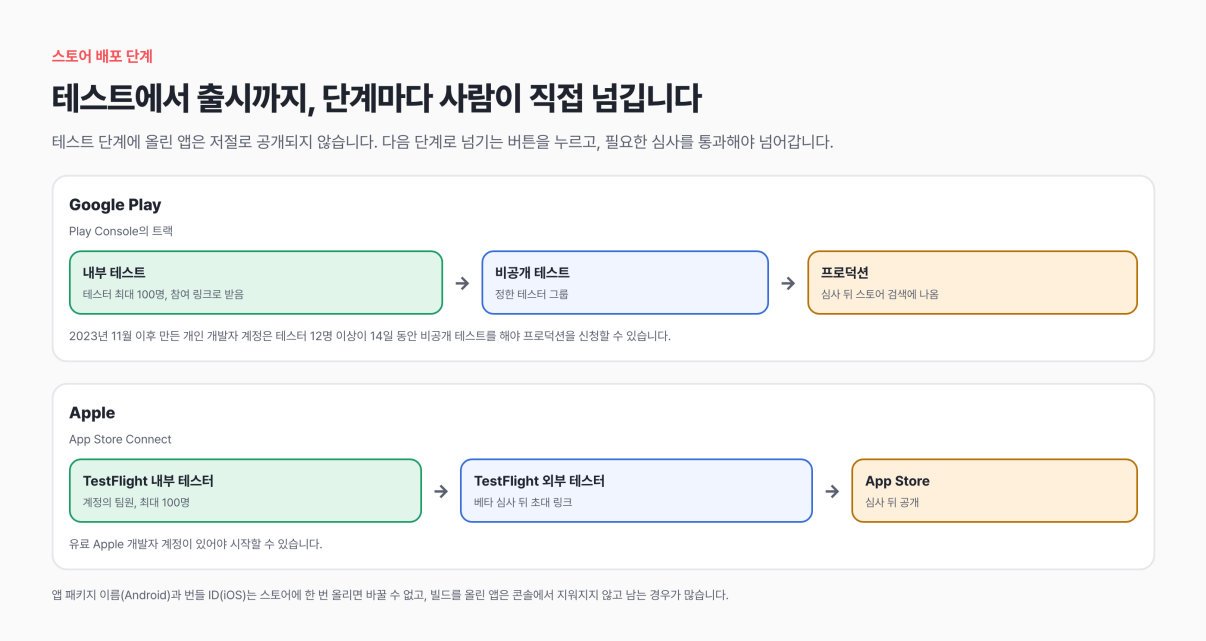


그림: Google Play와 Apple의 테스트에서 출시까지 단계

Google Play의 내부 테스트는 등록한 테스터(최대 100명)만 참여 링크로 받는 단계라서, 처음 앱을 올려 실제 스토어 경로로 설치해 보기에 알맞습니다. 앱의 첫 파일은 Play Console 웹에서 직접 올립니다. EAS Submit으로 자동으로 올리려면 Google 서비스 계정 키 파일(JSON)이 필요한데, 키 파일은 유출되면 그 계정으로 무엇이든 할 수 있어서 이 과정에서는 만들지 않습니다. 2023년 11월 이후 만든 개인 개발자 계정은 테스터 12명 이상이 14일 동안 비공개 테스트를 해야 프로덕션 출시를 신청할 수 있으므로, 하루 이틀 안에 정식 출시까지 가는 것은 어렵습니다.

Apple은 TestFlight로 테스트합니다. 계정의 팀원은 내부 테스터로 바로 받을 수 있고, 외부 테스터에게 보내려면 베타 심사를 거칩니다. 스토어에 한 번 올린 패키지 이름과 번들 ID는 바꿀 수 없고, 빌드를 올린 앱은 콘솔에서 지워지지 않고 남는 경우가 많습니다. 그래서 연습으로 올릴 때도 이름을 정해 두고 올리고, 테스트가 끝나면 테스트 트랙을 일시중지하거나 테스터를 빼서 정리합니다.

AI 서비스 앱의 출시 준비 점검

스토어는 앱이 사용자 데이터를 어떻게 다루는지와 앱 안에서 무엇을 파는지를 심사합니다. AI 서비스 앱은 여기에 AI가 만든 콘텐츠를 어떻게 다루는지가 더해집니다. 아래 표는 에이전트 서비스 앱을 기준으로 출시 전에 점검할 항목입니다.

항목	스토어가 보는 것	에이전트 서비스 앱에서
개인정보처리방침	어떤 데이터를 모으고 어디에 보내는지 적은 공개 주소	대화, 캐릭터, 기억과 약속을 저장하고, 대화를 외부 LLM API로 보낸다는 것을 적어야 함. 아직 없음
데이터 안전 항목(Google Play), 개인정보 항목(Apple)	수집하는 데이터 종류와 제3자 전송 여부를 스토어 양식에 답함	외부 LLM API로 보내는 대화도 제3자 전송에 해당함
계정과 데이터 삭제	앱 안에서 계정과 데이터를 지울 수 있는지	백엔드에 내 데이터 전체 삭제 API(`DELETE /api/data`)가 있지만 앱 화면에는 버튼이 없음
AI 생성 콘텐츠 신고	사용자가 부적절한 AI 답을 신고할 수 있는지	답마다 신고 버튼이 필요함. 아직 앱에 없음
AI임을 밝히기	사람인 척하지 않는지	캐릭터가 사람이라고 주장하지 않고, 화면에 AI 캐릭터임을 표시함
앱 안 판매	디지털 상품을 앱 안에서 팔면 스토어 결제를 써야 함	AI 크레딧을 앱 안에서 팔면 스토어 결제 정책이 적용됨. 결제는 웹에서 하고 앱에서는 사용만 하는 구조로 나누는 경우가 많음

항목	스토어가 보는 것	에이전트 서비스 앱에서
연령 등급	내용에 맞는 등급 설문	대화형 AI는 사용자가 만든 내용과 AI 답이 섞이므로 설문에 그대로 답함

표의 "아직 없음"은 출시 전에 채울 일입니다. 스토어 정책은 자주 바뀌므로, 실제로 올릴 때는 Google Play 정책 센터와 Apple App Review 지침의 최신 내용을 다시 확인합니다.

직접 확인할 곳: 내 앱의 출시 준비 점검표 만들기

1. 저장소에 `docs/store-readiness.md` 를 만들고 위 표의 항목을 옮깁니다.
2. "내 앱에서" 칸을 지금 내 앱 기준으로 채웁니다. 있으면 어느 화면에 있는지, 없으면 무엇을 만들어야 하는지 적습니다.
3. 앱이 모으는 데이터를 하나씩 적고, 각각이 어디(내 백엔드, Supabase, LLM 제공사)로 가는지 옆에 적습니다.
4. 후반부 직접 해 보기에서 이 표의 빈칸 하나를 채우는 일을 골라도 됩니다.

코딩 에이전트에게 맡길 때: 앱이 보내는 데이터 찾기

```
이 앱 코드와 src/lib/api.ts 를 읽고, 앱이 사용자에게서 받아 어디로 보내는 데이터를 모두 표로 정리해 줘. 열은 데이터, 받는 화면, 보내는 곳(API 경로), 그 뒤 백엔드에서 어디에 저장되거나 전달되는지(모르면 모른다고)로 해 줘. 코드는 바꾸지 마.
```

에이전트가 만든 표는 출발점입니다. 백엔드에서 데이터가 어디로 가는지는 앱 코드만으로 알 수 없으므로, "모른다"고 나온 칸은 백엔드 저장소의 코드와 문서를 보고 직접 채웁니다.

객관식 퀴즈

Q. 에이전트 서비스 앱에서 AI 크레딧을 팔고 싶습니다. 스토어 결제 정책을 피하면서 정책을 지키는 구조로 가장 알맞은 것은 무엇인가요?

- 1. 앱 안에 웹 결제 페이지를 띄워 크레딧을 판다
- 2. 크레딧 구매는 웹 서비스에서 하고, 앱에서는 같은 계정의 크레딧을 사용만 하게 한다
- 3. 앱 안에서 팔되 스토어 심사 때만 결제 버튼을 숨긴다
- 4. 크레딧이라고 부르지 않고 후원이라고 부른다**제출**

- •
  1. 앱 안에 웹 결제 페이지를 띄워 크레딧을 판다
- •
  - . 크레딧 구매는 웹 서비스에서 하고, 앱에서는 같은 계정의 크레딧을 사용만 하게 한다

- •

  3. 앱 안에서 팔되 스토어 심사 때만 결제 버튼을 숨긴다

- •

  4. 크레딧이라고 부르지 않고 후원이라고 부른다

📚 스토어 출시 참고 자료

- Google Play Console: 앱 테스트 설정: 내부 테스트, 비공개 테스트, 프로덕션 트랙의 차이와 테스터 등록 방법입니다.
- Google Play 정책 센터: 데이터 안전, AI 생성 콘텐츠, 결제 정책의 최신 내용을 확인할 수 있습니다.
- App Review 지침: Apple 심사 기준입니다. 앱 안 결제(3.1)와 사용자 생성 콘텐츠(1.2) 항목을 보면 됩니다.
- Expo: Submit to app stores: EAS Submit으로 스토어에 제출하는 흐름과 필요한 계정 조건이 나옵니다.

Chapter06

직접 해 보기의 규칙

섹션 4에서 템플릿 앱을 빌드해 폰에 설치했다면, 이번에는 내가 만들고 싶었던 앱을 Expo로 새로 만들어 같은 길을 끝까지 가 봅니다. 떠오르는 아이디어가 없다면 산책 기록, 냉장고 유통기한 메모, 영수증 가계부처럼 사진이나 위치 같은 기기 기능을 하나 쓰는 작은 앱이 하루 안에 끝내기 좋습니다. 코드는 코딩 에이전트에게 맡겨도 되지만, 화면 목록과 데이터가 어디서 오는지 같은 구조는 내가 먼저 정합니다. 힌트는 접혀 있으니 20분쯤 혼자 찾아본 뒤에 엽니다.

스테이지	할 일	클리어 확인
0. 앱 계획	아래 표를 내 앱으로 채워 `docs/app-plan.md` 에 둠	표만 보여 주고 옆 사람에게 1분 안에 설명함
1. 화면 만들기	`npx create-expo-app@latest` 로 새 프로젝트를 만들고 표의 화면을 만듦	Expo Go에서 모든 화면을 오가고, `npx tsc --noEmit` 과 `npx expo-doctor` 가 오류 없이 끝남
2. 기기 기능 하나	카메라, 위치, 로컬 알림 가운데 하나를 붙임	정한 때에 권한 창이 뜨고, 거부해도 앱이 멈추지 않음

스테이지	할 일	클리어 확인
3. 빌드와 설치	`preview` 프로필로 Android 빌드	노트북의 개발 서버를 끈 상태에서 설치한 앱이 Expo Go에서와 같이 돎
4. 출시 준비 점검표	섹션 5의 `docs/store-readiness.md` 를 내 앱으로 채움	모든 항목에 있음 또는 없음이 적혀 있고, 빈칸 하나를 실제로 채움

빌드는 무료 플랜의 월 횟수를 쓰므로 스테이지 1~2는 Expo Go로만 확인하고, 빌드는 스테이지 3에서 한 번에 합니다. iPhone만 있는 수강생은 스테이지 2까지 Expo Go로 확인하고, 스테이지 3은 짝의 Android 폰에 내 앱의 설치 링크를 보내 설치해 봅니다.

앱 계획 표

정할 것	예: 산책 기록 앱
앱이 하는 일 한 문장	산책한 날의 사진과 장소를 남기고 목록으로 다시 본다
화면 목록	기록 목록, 새 기록, 기록 보기
데이터가 오는 곳과 저장	폰 안 저장소(`@react-native-async-storage/async-storage`). 서버 없음. 앱을 지우면 기록도 사라짐
기기 기능과 Expo Go 지원	카메라(`expo-image-picker`), 위치(`expo-location`). 둘 다 Expo Go에서 됨

데이터를 어디에 둘지가 가장 큰 결정입니다. 폰 안에만 두면 서버가 필요 없지만 앱을 지우거나 폰을 바꾸면 데이터도 사라지고, 서버에 두면 백엔드가 있어야 합니다. 하루 안에 끝내려면 화면은 세 개 이하로, 데이터는 폰 안 저장이나 이미 있는 백엔드로 정하는 편이 안전합니다. 기기 기능이 Expo Go에서 되는지는 각 라이브러리 문서 위쪽의 "Included in Expo Go" 표시로 확인합니다.

> 📷 **캡처 필요** `cap_D77_02_new_screen_expo_go.png`
>
> 화면: 내 폰의 Expo Go
>
> 보여 줄 것: 새로 만든 화면이 Expo Go에서 열린 모습

▼ 힌트: 화면과 기기 기능

새 프로젝트에 모바일 템플릿의 `AGENTS.md` 와 `.claude/settings.json` 을 복사해 두면 같은 작업 규칙으로 에이전트를 쓸 수 있습니다. 화면이 안 보이면 파일이 `src/app/` 아래에 있는지, 탭 화면을 바꿨다면 `src/app/(tabs)/_layout.tsx` 에도 등록되어 있는지 봅니다. 권한을 한 번 거부하면 운영체제가 같은 창을 다시 띄우지 않는 경우가 있어서, 거부한 사용자에게는 폰 설정에서 권한을 켜는 방법을 안내합니다. 알림은 앱이 예약하는 로컬 알림만 Expo Go에서 되고, 서버가 보내는 푸시는 Android의 Expo Go에서 빠졌습니다.

▼ **힌트: 빌드**

새 프로젝트에는 `eas.json` 이 없으므로 `npx eas-cli@latest build:configure` 가 만들어 줍니다. 만들어진 `preview` 프로필이 Android를 APK로 만드는지(`"android": { "buildType": "apk" }`)와, 앱이 쓰는 `EXPO_PUBLIC_` 값이 프로필의 `env` 에 있는지 확인합니다. 빌드가 실패하면 빌드 로그에서 처음 실패한 단계를 보고, 코딩 에이전트에게 "npx eas-cli@latest 로 가장 최근 빌드가 왜 실패했는지 로그로 설명해 줘"라고 물어볼 수도 있습니다. 같은 오류로 두 번 빌드하지 않게, 고친 뒤 `npx expo-doctor` 를 먼저 돌립니다.

주관식 퀴즈

Q. [이해 점검] Expo Go에서 카메라 권한 창을 확인했는데, 빌드해서 설치한 내 앱에서는 권한 창의 앱 이름과 설명 문구가 다르게 떴습니다. 왜 다르게 보이는지, 출시 전에 이 문구를 어디서 정하고 무엇을 적어야 하는지 설명해 보세요.

-
  - /300

다음

서로 보여 주기

옆 사람과 짝을 지어 5분씩 내 앱을 보여 줍니다. 노트북을 닫은 상태에서 설치한 앱을 열고(iPhone이라면 Expo Go나 짝의 Android 폰으로), 화면을 하나씩 넘기며 앱 계획 표와 맞는지 봅니다. 듣는 사람은 "이 화면의 데이터는 어디서 오는가"와 "코드를 고치면 설치한 앱에 반영하려면 무엇을 해야 하는가"를 물어봅니다. 보여 주기 전에 앱 코드와 `eas.json` 에 시크릿이 없는지, `.env` 와 서명 키 파일이 저장소에 없는지 한 번 더 봅니다.

주관식 퀴즈

Q. [직접 해 보기 회고] 오늘 가장 오래 막힌 곳을 하나 골라, 무엇을 보고 막힌 줄 알았는지, 원인을 어디서 찾았는지, 어떻게 풀었는지 순서대로 적어 보세요.

-
  - /300

다음

📚 직접 해 보기 참고 자료

- Expo: Create a project: 새 프로젝트를 만드는 명령과 기본 템플릿 구조입니다.
- Expo: Notifications: 로컬 알림 예약과 권한 요청, Expo Go에서 되는 것과 안 되는 것이 나옵니다.
- Expo: Troubleshooting build errors: 빌드가 실패했을 때 로그를 읽는 순서입니다.

Chapter07

오늘 배운 것

개발 실행, 빌드, 제출. Expo Go는 개발 서버에서 코드를 받아 바로 바뀌지만 내 폰에서만 열리고, EAS Build는 클라우드에서 앱 파일을 만들어 링크로 나눠 줄 수 있게 하며, 스토어 배포는 개발자 계정으로 파일을 올려 테스트와 심사를 거칩니다. 빌드한 앱에는 코드가 고정되므로 고치면 다시 빌드해야 하고, 빌드에 들어갈 값은 `.env` 가 아니라 `eas.json` 에 적습니다.

한 번 정하면 바꾸기 어려운 것. 패키지 이름과 서명 키는 스토어가 앱을 알아보는 기준이라 한 번 올리면 바꾸기 어렵습니다. 서명 키는 EAS가 보관하게 해 키 파일이 내 컴퓨터와 저장소에 남지 않게 했습니다.

출시 준비. 스토어는 개인정보처리방침, 데이터 안전 항목, 계정과 데이터 삭제, AI 생성 콘텐츠 신고, 앱 안 판매를 심사합니다. 테스트 단계에 올린 앱은 사람이 넘기고 심사를 통과하기 전에는 공개되지 않습니다.

내 앱 만들기. 화면 목록과 데이터가 오는 곳을 먼저 정한 뒤 에이전트에게 구현을 맡겼고, 기기 기능 하나를 붙여 빌드하고 설치했습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 Expo Go로 실행하는 앱과 스토어에서 받은 앱의 차이를 적었습니다. 그 답을 다시 읽고, 오늘 직접 본 것(노트북 없이 열린 앱, 고친 코드가 반영되지 않은 설치 앱, 새 게스트로 시작한 앱, 스토어 배포 단계)과 비교해 맞았던 것과 새로 알게 된 것을 적어 보세요.